

Specification-Driven Verification Architecture

Purpose

This document defines the architecture for integrating:

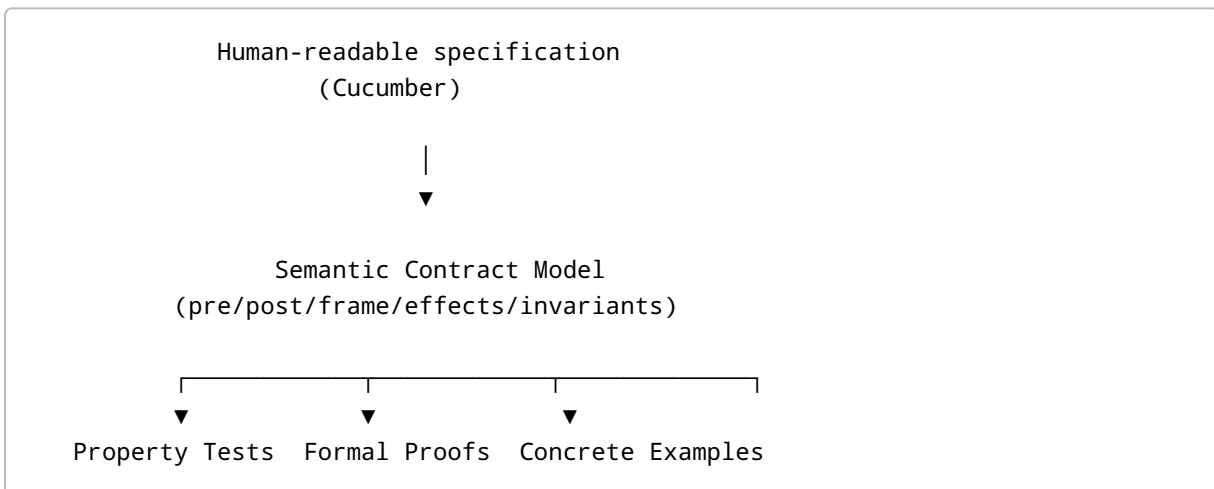
- Cucumber/Gherkin
- Formal contracts
- Property-based testing
- Concrete example testing
- Formal verification

The objective is that **there is exactly one semantic specification** from which all testing and verification artefacts are derived.

The system must never evolve separate specifications for testing and verification.

Fundamental Principle

There must only be one source of semantic truth.



The Cucumber specification is **not** itself the formal specification.

Likewise, the contracts are **not** independent of Cucumber.

Instead:

- Cucumber expresses behavioural intent.
- Every behavioural statement resolves to a semantic proposition.

- Every consumer (testing, proof, documentation) interprets those same propositions.
-

Design Goals

The architecture shall support:

1. Human-readable behavioural specifications.
 2. Machine-readable semantic contracts.
 3. Property-based fuzz testing.
 4. Concrete executable scenarios.
 5. Formal verification.
 6. Specification sanity testing.
 7. Counterexample recording.
 8. Traceability between all layers.
-

Architectural Layers

Layer 1 — Behaviour Specification

This consists of ordinary Cucumber/Gherkin.

Example:

Feature: Funds transfer

Rule: Successful transfers preserve total funds

Scenario: Transfer between two accounts

Given an account with sufficient balance

When funds are transferred

Then the total balance is unchanged

This layer should remain understandable by domain experts.

It intentionally avoids implementation details.

It should avoid mathematics wherever possible.

Layer 2 — Semantic Contract

Every normative sentence in Layer 1 must resolve to one or more semantic propositions.

Example

```
transfer.pre.valid_amount  
  
transfer.post.total_preserved  
  
transfer.post.source_decreased  
  
transfer.post.target_increased
```

Each proposition has an executable contract definition.

Example

```
@predicate  
def total_balance_preserved(old_source,  
                             old_target,  
                             new_source,  
                             new_target):  
  
    return (  
        old_source.balance + old_target.balance  
        ==  
        new_source.balance + new_target.balance  
    )
```

This layer is the semantic truth.

Everything below consumes this.

Nothing below invents additional semantics.

Contract Language

Contracts shall be written in a pure subset of Python.

The contract language must satisfy the following properties.

Pure

Contracts shall

- perform no mutation
 - perform no I/O
 - have no global state
 - be deterministic
-

Executable

Contracts shall execute directly inside Python.

This allows:

- runtime assertion checking
 - property testing
 - debugging
 - specification testing
-

Inspectable

Contracts must have a syntax tree suitable for

- SMT translation
 - proof generation
 - optimisation
 - simplification
 - dependency tracking
-

Rich enough for verification

The language must eventually support

- forall
- exists
- implication
- old(...)
- unchanged(...)
- modifies(...)
- ownership
- frames

- effects
 - collections
 - sequences
 - maps
 - finite sets
-

Preconditions and Postconditions

Every operation consists of

```
Pre(state, input)
```

and

```
Post(old_state,  
      input,  
      result,  
      new_state)
```

Frame conditions and effect specifications are separate semantic objects.

They are not hidden inside arbitrary code.

Specification Registry

Every semantic proposition shall have a stable identifier.

Example

```
transfer.pre.valid_amount  
  
transfer.post.total_preserved  
  
transfer.frame.account_only  
  
transfer.effect.audit_event
```

The registry provides:

- identifier
- implementation
- documentation
- dependencies
- proof status
- testing status

Cucumber references identifiers.

Testing references identifiers.

Formal proofs reference identifiers.

Gherkin Binding

A Gherkin sentence is not implemented directly in Python.

Instead it resolves to a semantic proposition.

Example

```
Then the total balance is unchanged
```

maps to

```
transfer.post.total_preserved
```

The Python step implementation merely invokes the semantic proposition.

No additional assertions are permitted.

The semantic proposition remains the single source of truth.

Semantic Refinement

A specification may be refined.

Refinement means

increasing semantic precision.

Example

```
Transfers preserve funds.
```

may refine into

```
source decreases  
target increases  
total preserved  
currency unchanged  
account identity unchanged
```

Each refinement introduces stronger obligations.

Refinement changes meaning.

Refinement is therefore a semantic operation.

Realisation

Realisation is distinct from refinement.

Realisation connects an abstract specification to an implementation.

Example

```
transfer.post.total_preserved
```

may be realised by

```
payments.service.transfer()
```

or

```
BankTransferService.execute()
```

Each implementation shares the same semantic contract.

Realisation does not change meaning.

It only changes where the specification is evaluated.

Contract Entry Points

Formal verification targets contract entry points.

A contract entry point is

- function
- method
- service operation
- API endpoint

that implements a semantic contract.

The contract entry point is the verification boundary.

Observation Seams

Testing requires observation seams.

An observation seam provides

- dependency substitution
- state extraction
- event capture
- mocking
- isolation

Observation seams belong to testing.

They are not part of the semantic specification.

Test Realisations

A test realisation contains

- implementation binding
- fixture construction
- observation seam
- state adapters
- execution strategy

This allows the same specification to test

- production code
- mocks
- simulators
- database implementations

without changing semantics.

Property-Based Testing

Property testing consumes semantic contracts.

It does not consume Gherkin directly.

The workflow is

Contract

↓

Input domain

↓

Generator

↓

Execution

↓

Postcondition evaluation

Input Domains

A contract defines legal inputs.

Example

```
amount > 0  
  
amount <= source.balance
```

This domain must support multiple interpretations.

Logical Interpretation

Predicate.

Generator Interpretation

Produce values satisfying the predicate.

Shrinker Interpretation

Reduce failing inputs while remaining inside the predicate.

Solver Interpretation

Translate to SMT.

Proof Interpretation

Become hypotheses during verification.

Automatic Generation

The system should support automatic generation whenever possible.

However:

Not every arbitrary predicate is generative.

Example

$$x^2 + y^2 == 1729$$

is perfectly valid logically but is not naturally invertible.

Therefore generators should be

- explicit
- synthesised where possible
- solver-backed where necessary

The architecture must not assume all predicates are invertible.

Concrete Examples

Concrete examples are **not** refinements.

Concrete examples are instantiations.

Example

```
balance = 100
```

```
amount = 20
```

satisfies

```
amount <= balance
```

Examples serve as

- documentation
- regression tests
- sanity checks
- demonstrations
- boundary cases

Scenario Instances

An executable scenario consists of

```
Abstract scenario
+
Implementation binding
+
Observation seam
+
Concrete values
```

This is a scenario instance.

Specification Testing

The specification itself must be testable.

This is distinct from testing implementations.

The specification should be executable independently.

Positive Tests

Generate values satisfying

Pre

Execute implementation.

Check

Post

Negative Tests

Generate values violating one precondition.

Verify

- rejection
- exception
- error handling

Postcondition Witnesses

Construct before/after states satisfying

Post

without executing implementation.

Inspect them manually.

This detects under-specification.

Specification Mutation

Generate outputs violating exactly one postcondition clause.

Ensure each clause is meaningful.

Counterexample Preservation

Every failing property test shall be minimised.

The minimal counterexample becomes a permanent scenario.

Example

Scenario: Regression #431

Given balance = 1

And amount = 1

...

Counterexamples therefore become documentation.

Traceability

Every artefact must be traceable.

Example

Feature

↓

Rule

↓

Scenario

↓

Semantic Proposition

↓

Contract

↓

Implementation

↓

Property Tests

↓

Proof Obligations

↓

Concrete Examples

No duplicated specifications are permitted.

Formal Verification

Verification proves that an implementation satisfies semantic contracts.

The proof should establish

Pre

⇒

Execution

⇒

Post

using the implementation's abstraction relation.

The proof consumes the same semantic contract used during testing.

No verification-specific semantics should exist.

Required Project Structure

```
features/  
contracts/  
spec_registry/  
generators/  
proofs/  
bindings/  
tests/  
examples/  
counterexamples/
```

Dependency Rules

Allowed

```
Feature  
↓  
Semantic Contract  
↓  
Testing  
↓  
Proof
```

Forbidden

Feature

↓

Testing semantics

Feature

↓

Proof semantics

The semantic contract must always sit between behavioural specifications and consumers.

AI Development Rules

The AI shall always obey the following.

Never duplicate specifications.

If two descriptions express the same property, they must reference the same semantic proposition.

Never hide semantics inside test glue.

Cucumber step implementations may only invoke semantic propositions.

They may not invent assertions.

Never hide semantics inside proofs.

Proofs prove existing contracts.

They do not redefine behaviour.

Keep behaviour abstract.

Gherkin should describe behaviour.

Not implementation.

Separate refinement from instantiation.

Refinement

- strengthens meaning
- decomposes specifications

Instantiation

- provides values

These are different operations.

Separate refinement from realisation.

Realisation attaches a specification to an implementation.

It does not strengthen semantics.

Record every counterexample.

Counterexamples become permanent executable scenarios.

Specifications are executable artefacts.

Contracts should support

- execution
- property testing
- solver translation
- formal verification
- documentation

without rewriting.

Long-Term Objective

The long-term objective is a specification ecosystem in which:

- Cucumber remains the human-readable behavioural description.
- Semantic contracts provide the unique formal meaning.
- Property-based testing, concrete examples, and formal verification are all mechanically derived from the same semantic model.
- Every discovered counterexample strengthens the specification by becoming a permanent documented scenario.
- There is never more than one authoritative definition of system behaviour.